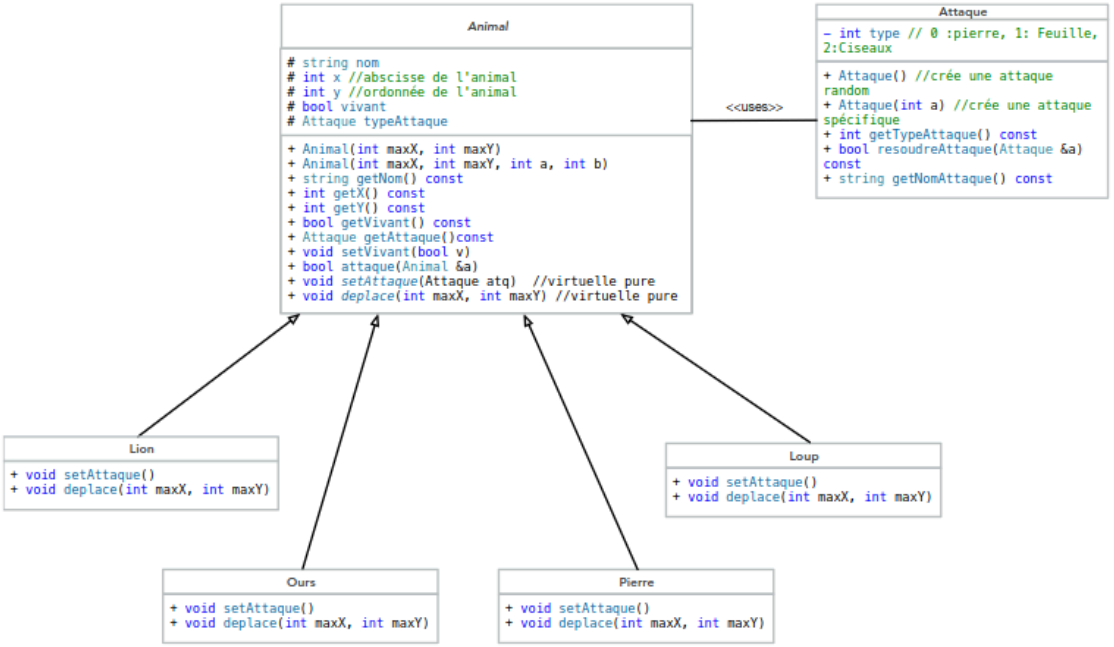
David Prospérin
Riana Chauvet Raherimalala

10/03/2026

Compte-rendu SMP-TP 8

1. Introduction

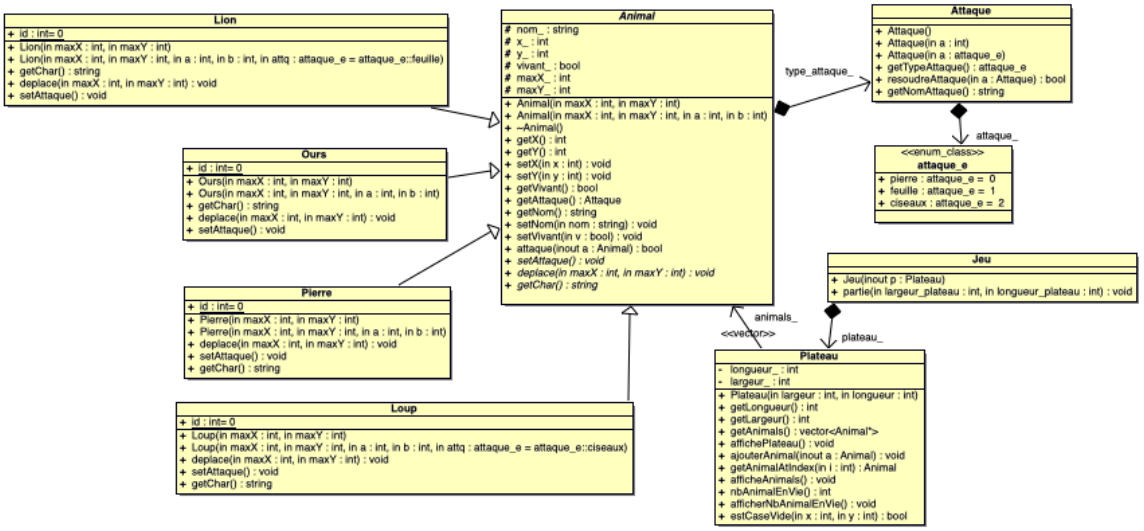
Ce TP a pour but de perfectionner notre utilisation de la programmation orientée objet en C++. Pour ce faire, on s'appuiera sur la situation proposée par le TP : le jeu de la vie. Dans ce jeu, des animaux se déplacent sur une grille et s'affrontent au "pierre, feuille, ciseaux" lorsqu'ils se retrouvent au même endroit dans la grille.
Pour réaliser ce TP, nous nous appuierons sur le diagramme de classe proposé par le sujet et nous avons ajouté les classes `Plateau` et `Jeu` qui nous permettra la gestion du plateau et, de manière plus générale, de la partie.
Voici le diagramme proposé :



Voici le diagramme de la classe ajoutée :

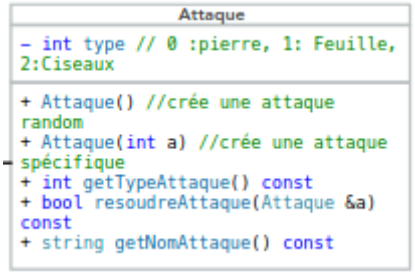
David Prospérin
Riana Chauvet Raherimalala

10/03/2026



2. La classe Attaque

Cette classe a pour objectif de gérer les attaques entre les différents animaux comme décrit sur le diagramme ci-dessous :



Voici les tests effectué pour vérifier le bon fonctionnement des méthodes de la classe `Attaque` :

```
cout << "-------TEST ATTAQUE RANDOM ET PRÉVUE-------" << endl;
cout << "Attaque random :" << endl;
cout << a1.getTypeAttaque() << endl;
cout << "Attaque prévue :" << endl;
cout << a2.getTypeAttaque() << endl;
cout << "-----------------------------------------" << endl;

cout << "-------TEST RESOLUTION ATTAQUE-------" << endl;
cout << "Attaque gagnante pour a2 :" << endl;
cout << a2.resoudreAttaque(a3) << endl;
cout << "Attaque perdante pour a2 :" << endl;
cout << a2.resoudreAttaque(a4) << endl;
cout << "Attaque avec égalité :" << endl;
cout << a2.resoudreAttaque(a5) << endl;
cout << "-----------------------------------------" << endl;
```

Et les résultats obtenus :

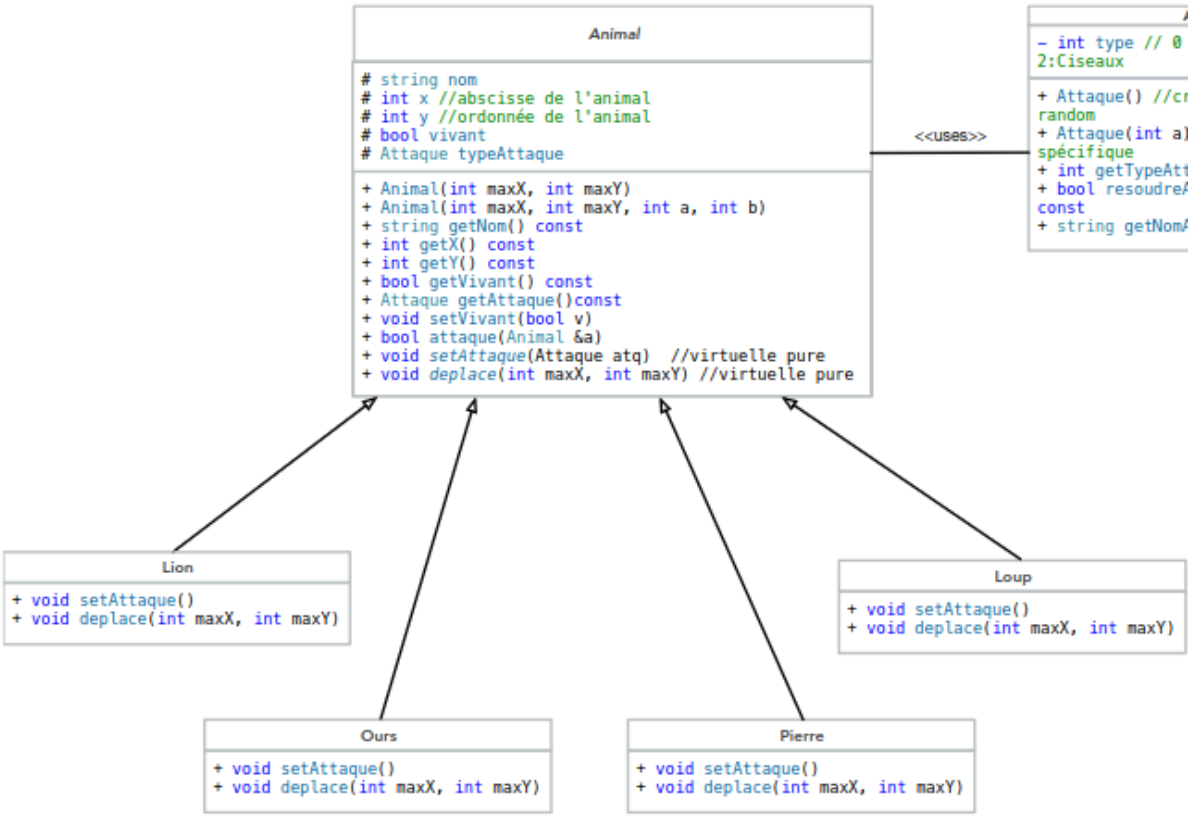
```
-------TEST ATTAQUE RANDOM ET PRÉVUE-------
Attaque random :
pierre
Attaque prévue :
ciseaux
-----------------------------------------
-------TEST RESOLUTION ATTAQUE-------
Attaque gagnante pour a2 :
1
Attaque perdante pour a2 :
0
Attaque avec égalité :
0
-----------------------------------------
```

3. La classe Animal

Cette classe a pour objectif de paramétrer les animaux. De cette classe découlent des classes filles pour chacun des animaux présents sur le plateau. Voici le diagramme de ces classes :

David Prospérin
Riana Chauvet Raherimalala

10/03/2026



Nous avons réalisé des tests afin de vérifier le bon fonctionnement de la classe `Animal` et de ses classes filles.

```
// Test attaque
cout << "-------Test attaque --------" << endl;
auto lion2   = Lion(maxX, maxY);
auto loup2   = Loup(maxX, maxY);
auto loup3   = Loup(maxX, maxY);
auto loup4   = Loup(maxX, maxY);
auto ours2   = Ours(maxX, maxY);
auto ours3   = Ours(maxX, maxY);
auto pierre3 = Pierre(maxX, maxY);
auto pierre4 = Pierre(maxX, maxY);

// ------------------------------------------------------------------------------

auto attq_feuille_pierre = lion2.attaque(pierre3);
assert(attq_feuille_pierre == true && "La feuille gagne contre la pierre");
assert(lion2.getVivant() == true && "Le lion doit être vivant après l'attaque");
assert(pierre3.getVivant() == false && "La pierre doit être morte après l'attaque");

auto attq_ciseaux_pierre = loup2.attaque(pierre4);
assert(attq_ciseaux_pierre == false && "Les ciseaux perdent contre la pierre");
assert(loup2.getVivant() == false && "Le loup doit être mort après l'attaque");
```

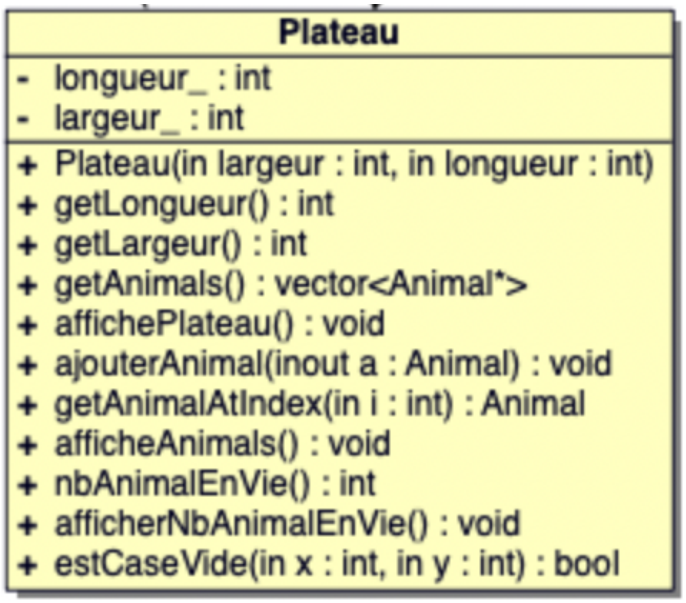
```
assert(pierre4.getVivant() == true && "La pierre doit être vivante après l'attaque");

// ------------------------------------------------------------------------------

auto attq_pierre_ciseaux = pierre4.attaque(loup3);
assert(attq_pierre_ciseaux == true && "La pierre gagne contre les ciseaux");
assert(pierre4.getVivant() == true && "La pierre doit être vivante après l'attaque");
assert(loup3.getVivant() == false && "Le loup doit être mort après l'attaque");

auto attq_feuille_ciseaux = ours2.attaque(loup4);
assert(attq_feuille_ciseaux == false && "La feuille perd contre les ciseaux");
assert(ours2.getVivant() == false && "L'ours doit être mort après l'attaque");
assert(loup4.getVivant() == true && "Le loup doit être vivant après l'attaque");

// ------------------------------------------------------------------------------

auto attq_pierre_feuille = pierre4.attaque(ours3);
assert(attq_pierre_feuille == false && "La pierre perd contre la feuille");
assert(pierre4.getVivant() == false && "La pierre doit être morte après l'attaque");
assert(ours3.getVivant() == true && "L'ours doit être vivant après l'attaque");
auto attq_ciseaux_feuille = loup4.attaque(ours3);
assert(attq_ciseaux_feuille == true && "Les ciseaux gagnent contre la feuille");
assert(loup4.getVivant() == true && "Le loup doit être vivant après l'attaque");
assert(ours3.getVivant() == false && "L'ours doit être mort après l'attaque");

// ------------------------------------------------------------------------------
// Attaque un animal vivant vs en animal mort

auto attq_vivant_mort = loup4.attaque(ours3);
assert(attq_vivant_mort == false && "Un animal vivant ne peut pas attaquer un animal mort");
assert(loup4.getVivant() == true && "Le loup doit être vivant après l'attaque");
assert(ours3.getVivant() == false && "L'ours doit être mort après l'attaque");

auto attq_mort_vivant = ours3.attaque(loup4);
assert(attq_mort_vivant == false && "Un animal mort ne peut pas attaquer un animal vivant");
assert(ours3.getVivant() == false && "L'ours doit être mort après l'attaque");
assert(loup4.getVivant() == true && "Le loup doit être vivant après l'attaque");

auto attq_mort_mort = pierre4.attaque(ours3);
assert(attq_mort_mort == false && "Un animal mort ne peut pas attaquer un animal mort");
assert(pierre4.getVivant() == false && "La pierre doit être morte après l'attaque");
assert(ours3.getVivant() == false && "L'ours doit être mort après l'attaque");
```

4. La classe Plateau

Nous avons choisi d'ajouter cette classe en plus de celle déjà présente dans le TP afin de pouvoir regrouper toutes les méthodes qui nous permettent la gestion du plateau.
Voici le diagramme son diagramme de classe :

David Prospérin
Riana Chauvet Raherimalala

10/03/2026



Nous avons réalisé des tests afin de vérifier le bon fonctionnement du plateau :
D'abord, nous avons essayé d'afficher un plateau de taille 5*6 vide.

```
// Test plateau
const int maxX = 5;
const int maxY = 6;
auto p = Plateau(maxX,maxY);

// Affichage du plateau vide
p.affichePlateau();
```

Nous obtenons bien le résultat attendu

```
|----|----|----|----|----|
|    |    |    |    |    |
|----|----|----|----|----|
|    |    |    |    |    |
|----|----|----|----|----|
|    |    |    |    |    |
|----|----|----|----|----|
|    |    |    |    |    |
|----|----|----|----|----|
|    |    |    |    |    |
|----|----|----|----|----|
|    |    |    |    |    |
|----|----|----|----|----|
```

Puis nous avons essayé d'ajouter des animaux sur dans notre plateau.

David Prospérin
Riana Chauvet Raherimalala

10/03/2026

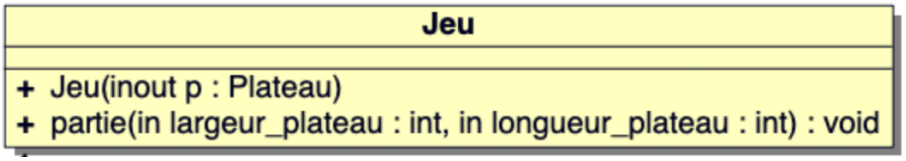
```
p.ajouterAnimal(new Loup(maxX, maxY, 1, 1));
p.ajouterAnimal(new Lion(maxX, maxY, 2, 2));
p.ajouterAnimal(new Ours(maxX, maxY, 3, 4));
p.ajouterAnimal(new Pierre(maxX, maxY, 2, 3));

p.ajouterAnimal(new Pierre(maxX, maxY, 5, 6));
assert(p.getAnimalAtIndex(4)->getX() == 0 && "Position x réelle de la pierre doit être 0");
assert(p.getAnimalAtIndex(4)->getY() == 0 && "Position y réelle de la pierre doit être 0");

p.getAnimalAtIndex(3)->setX(6);
assert(p.getAnimalAtIndex(3)->getX() == 1 && "Position x réelle de la pierre doit être 1");

p.getAnimalAtIndex(1)->setY(6);
assert(p.getAnimalAtIndex(1)->getY() == 0 && "Position y réelle du lion doit être 0");

// Affichage du plateau avec un Loup, un Lion, un Ours et une Pierre
cout << "-------Plateau : Loup, Lion, Ours et Pierre -------" << endl;
p.affichePlateau();
p.afficherNbAnimalEnVie();

p.afficheAnimals();
```

Nous obtenons ainsi :

```
-------Plateau : Loup, Lion, Ours et Pierre -------
|----|----|----|----|----|
| 🗿 |    | 🦁 |    |    |
|----|----|----|----|----|
|    | 🐺 |    |    |    |
|----|----|----|----|----|
|    |    |    |    |    |
|----|----|----|----|----|
|    | 🗿 |    |    |    |
|----|----|----|----|----|
|    |    |    | 🐻 |    |
|----|----|----|----|----|
|    |    |    |    |    |
|----|----|----|----|----|
Il y a 5 animaux vivants
```

5. La classe Jeu

De même que pour la classe `Plateau`, nous avons choisi d'implémenter une classe `Jeu` destinée à faire tourner les parties. Voici le diagramme de classe associé :



Pour chaque partie, nous laissons le choix de la dimension du plateau à l'utilisateur. En ce qui concerne les différents animaux à placer sur le plateau, nous remplissons 25% des

cases du plateau avec des animaux dont le nombre d'individus par espèces est réparti le plus équitablement possible.

Conclusion

En conclusion, ce TP a permis de mettre en pratique la programmation orientée objet en C++ à travers la modélisation d'un jeu sur une grille. Les classes Attaque, Animal, Plateau et Jeu permettent de gérer les interactions entre les animaux, l'état du plateau et le déroulement des parties. Les tests réalisés ont validé le fonctionnement des principales fonctionnalités, même si l'ensemble du programme n'a pas encore été entièrement testé.